

W3 PRACTICE

Express Basics + POST + Middleware

 *At the end of this practice, you can*

- ✓ **Create** and run a express.js HTTP server
- ✓ **Implement** route handling using express.js
- ✓ Parse form data from POST requests with middleware.
- ✓ Apply middleware concept to logging

 *Get ready before this practice!*

- ✓ **Read** the following documents to understand the nature of Express.js:
<https://expressjs.com/>
- ✓ **Read** the following documents to know more about Express.js's built-in middleware's:
<https://expressjs.com/en/resources/middleware.html>
- ✓ **Read** the following documents to understand MDN: HTTP POST:
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods/POST>
- ✓ **Read** the following documents to array filter:
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/filter

 *How to submit this practice?*

- ✓ Once finished, push your **code to GITHUB**
- ✓ Join the **URL of your GITHUB** repository on LMS



EXERCISE 1 – Refactoring

Goals

- ✓ Take advantage of Express.js framework's flexibility and minimalism
- ✓ Refactor code from node.js's built-in HTTP Module



Refactor the source code of EXERCISE 2 & 3 in Week 2 to Express.js

Q1 – What challenges did you face when using the native http module that Express.js helped you solve?

Challenges that I faced when using the native http module that Express.js helps solve:

- _ **Manual Routing:** must inspect `req.url` + `req.method` and branch it myself.
- _ **Parsing Inputs:** reading request body streams and decoding form/JSON is boilerplate and easy to make mistake.
- _ **Response Helpers:** setting headers/status/content-type consistently is repetitive
- _ **Error handling:** must wire `req.on('error')`, parsing errors, and per-route failures manually
- _ **Code structure/scaling:** lots of `if/switch` logic becomes hard to organize as routes grow

Q2 – How does Express simplify route handling compared to the native HTTP server?

Compared to the native HTTP server, Express simplify route handling by declaring routes directly by method + path (`app.get('/contact', ...)`, `app.post('/contact', ...)`) instead of using one handler for everything (`if (url === '/contact' && method === 'POST') ... else ...`). And Express has Built-in 404 flow, which just need to add one `app.use((req,res)=>res.status(404)...`) instead of sprinkling 404 returns if no routes matches.

Q3 – What does middleware mean in Express, and how would you replicate similar behavior using the native module?

In Express, Middleware means a function that run before route handler, calling `next()` passes control onward. To replicate similar behavior using the native module, I would build my own “pipeline” by running an array of functions in order, then finally routing.

EXERCISE 2 – API for Course Records

🔧 For this exercise you will start with a **START CODE (EX-2)**

Goals

- ✓ Understand Route Parameters (:param)
- ✓ Work with Query Parameters (?key=value)
- ✓ Implement Conditional Logic for Filtering
- ✓ Build Real-World Web API Behavior
- ✓ Practice Defensive Programming

Context

You are building a backend API for a university's course catalog. Each course has the following fields

```
{
  "id": "CSE101",
  "title": "Introduction to Computer Science",
  "department": "CS",
  "level": "undergraduate",
  "credits": 3,
  "instructor": "Dr. KimAng",
  "semester": "fall"
}
```

Q1 - Create a route

```
GET /departments/:dept/courses
```

EXAMPLE

```
/departments/CSE/courses
```

Q2 - Accept query parameters to filter the result:

- `level` → e.g., undergraduate, graduate
- `minCredits` → integer
- `maxCredits` → integer
- `semester` → fall, spring, etc.
- `instructor` → partial match

EXAMPLE

```
/departments/CSE/courses?level=undergraduate&minCredits=2&semester=fall
```

Q3 - Return a JSON array of courses that match:

- The `:dept` from the route parameter
- The filter criteria from query parameters

Q4 – Handle Edge Cases

- **Invalid credit ranges** (minCredits > maxCredits)
- **No matching courses**
- **Missing or unsupported** query parameters (ignore them silently)

EXAMPLES

REQUEST
/departments/CSE/courses?level=undergraduate&minCredits=3&instructor=KimAng
RESPONSE
<pre>{ "results": [{ "id": "CSE101", "title": "Introduction to Data Science", "department": "CSE", "level": "undergraduate", "credits": 3, "instructor": "Dr. KimAng", "semester": "fall" }], "meta": { "total": 1 } }</pre>

Test cases

- `http://localhost:3000/departments/CSE/courses`
- `http://localhost:3000/departments/CSE/courses?level=undergraduate`
- `http://localhost:3000/departments/CSE/courses?minCredits=4`
- `http://localhost:3000/departments/CSE/courses?instructor=smith&semester=fall`

EXERCISE 3 – Enhance an API with Middleware

Goal

Your goal is to modularize and secure your course filtering API using **Express middleware**. Middleware helps keep your code clean, reusable, and extensible.

Q1 - Create a middleware function that logs the following for every request:

- HTTP method (GET, POST, etc.)
 - Request path (e.g., /departments/CSE/courses)
 - Query parameters
 - Timestamp in ISO format
- ✓ **Apply this middleware globally** so it logs **all incoming requests** to the server.

Q2 - Create a route-specific middleware to **validate query parameters**:

- If minCredits or maxCredits are present, ensure they are valid integers.
 - If minCredits > maxCredits, return 400 Bad Request with an error message.
- ✓ **Apply this middleware only** to the /departments/:dept/courses route.

Q3 – (Bonus) Token-Based Authentication Middleware

Simulate basic API security:

- Require a token query parameter (e.g., ?token=xyz123)
 - If the token is missing or incorrect, respond with 401 Unauthorized.
- ✓ This middleware can be applied **either globally or to specific routes**.

Deliverables

- `logger.js` – contains your logging middleware.
- `validateQuery.js` – contains your validation middleware.
- `auth.js` (optional) – contains your token authentication middleware.
- `server.js` – where you apply middleware and define the course filtering route.

Test cases

```
GET /departments/CSE/courses?minCredits=abc
```

→ should return 400 Bad Request

```
GET /departments/CSE/courses?minCredits=4&maxCredits=2
```

→ should return 400 Bad Request

```
GET /departments/CSE/courses?token=xyz123
```

→ should succeed if token middleware is active

REFLECTIVE QUESTIONS

✎ For this part, submit it in separate PDF files

Middleware & Architecture

1. What are the advantages of using middleware in an Express application?

The advantages of using Middleware in an Express application is that it allows requests to be processed before reaching the main route logic. It is commonly used for tasks like authentication, logging, validation, and modifying request or response data. By reusing middleware across routes, the application becomes more structured, maintainable, and efficient. Middleware functions can also control the request-response cycle or forward execution using `next()`.

2. How does separating middleware into dedicated files improve the maintainability of your code?

Separating middleware into dedicated files helps organize the project more effectively. Each file can handle a specific task, such as authentication or request logging, making the code easier to read and maintain. This structure also simplifies debugging and collaboration since developers can update one feature without interfering with unrelated parts of the application.

3. If you had to scale this API to support user roles (e.g., admin vs student), how would you modify the middleware structure?

If I had to scale this API to support user roles, I would divide the middleware structure into authentication and authorization layers. One middleware would confirm the user's identity, while another would verify permissions such as admin or student access. Routes could then apply the appropriate middleware depending on the required access level, making the API easier to expand and manage.

Query Handling & Filtering

4. How would you handle cases where multiple query parameters conflict or are ambiguous (e.g., `minCredits=4` and `maxCredits=3`)?

In cases where multiple query parameters conflict or are ambiguous, The server should validate query parameters before applying filters. When conflicting values are detected, such as `minCredits` being higher than `maxCredits`, the API should return a `400 Bad Request` response with a clear explanation of the problem. This ensures the filtering logic remains accurate and predictable.

5. What would be a good strategy to make the course filtering more user-friendly (e.g., handling typos in query parameters like “falll” or “dr. smtih”)?

To make the course filtering more user-friendly, A good strategy would be implementing fuzzy search or similarity matching to recognize small spelling errors. The system could also ignore letter casing and remove extra spaces when processing searches. These enhancements would make filtering more flexible and provide a smoother experience for users.

Security & Validation

6. What are the limitations of using a query parameter for authentication (e.g., `?token=xyz123`)? What alternatives would be more secure?

The limitations of using a query parameter for authentication are tokens may be exposed in URLs, browser history, or server logs. A more secure alternatives would be using JWT tokens in Authorization headers, session-based authentication, or OAuth methods, which offer stronger protection for sensitive credentials.

7. Why is it important to validate and sanitize query inputs before using them in your backend logic?

It is important to validate and sanitize query inputs before using them in backend logic because it helps protect the application from invalid or harmful data. Without proper checks, dangerous input could cause system errors or security vulnerabilities such as SQL injection attacks. Ensuring data is cleaned and verified before processing improves both safety and reliability.

Abstraction & Reusability

8. Can any of the middleware you wrote be reused in other projects? If so, how would you package and document it?

Yes, middleware like authentication, validation, and request logging can be reused in others projects. I would organize them into standalone modules or publish them as npm packages with detailed setup instructions and usage examples. Proper documentation would make integration easier and reduce development time in future applications.

9. How could you design your route and middleware system to support future filters (e.g., course format, time slot)?

To support future filters, I would create a flexible filtering system where new query parameters can be added without major changes to the route logic. Middleware could handle validation and preprocessing of filters before passing the data to the controller. This modular approach would make future updates simpler and keep the API scalable.

Bonus – Real-World Thinking

- 10.** How would this API behave under high traffic? What improvements would you need to make for production readiness (e.g., rate limiting, caching)?

Under high traffic, the API could experience slower response times or server overload due to many simultaneous requests. To make production readiness, I would need to implement solutions such as rate limiting, caching, load balancing, and database optimization. Adding proper monitoring and error-handling systems would also improve overall stability and performance.